

# HSBC Email Automation

Automatización de procesamiento de emails de solicitudes de crédito para BTG.



**Build/run sin instalar nada (solo Docker):** ver [Build y ejecución](#).



**Estado de la migración y cómo validar cada fase:** [docs/Avance.md](#) · [docs/VALIDACION.md](#).

## Descripción General

Lee los correos **no leídos** de una casilla vía **Microsoft Graph API**, extrae datos estructurados del cuerpo mediante parseo de texto, y los envía a un Web Service **SOAP** de GeneXus (Bantotal) para dar de alta leads. Tras procesar cada correo lo marca como leído.

**Migración 3.0.0:** el proyecto migró de **IMAP/JavaMail** → **Microsoft Graph** y de **Java 8 / Tomcat 8.5** → **Java 21 (Azul Zulu) / Tomcat 11 (Jakarta EE 11)**. Ver [docs/migracion\\_graph\\_api.md](#).

## Requisitos

**Para desarrollar / buildear** (política cero-runtimes): **solo Docker**. El build corre en un build-container; no se instala Java ni Maven local. Ver el repo de infra hermano `docker-hsbc-email-autom`.

**Para producción** (deploy del WAR):

- **Java 21** (Azul Zulu)
- **Apache Tomcat 11.0.x**
- Una **App Registration** en Azure AD (Entra ID) con permiso `Mail.ReadWrite` (Application)
- Web Service SOAP de destino (GeneXus/Bantotal)

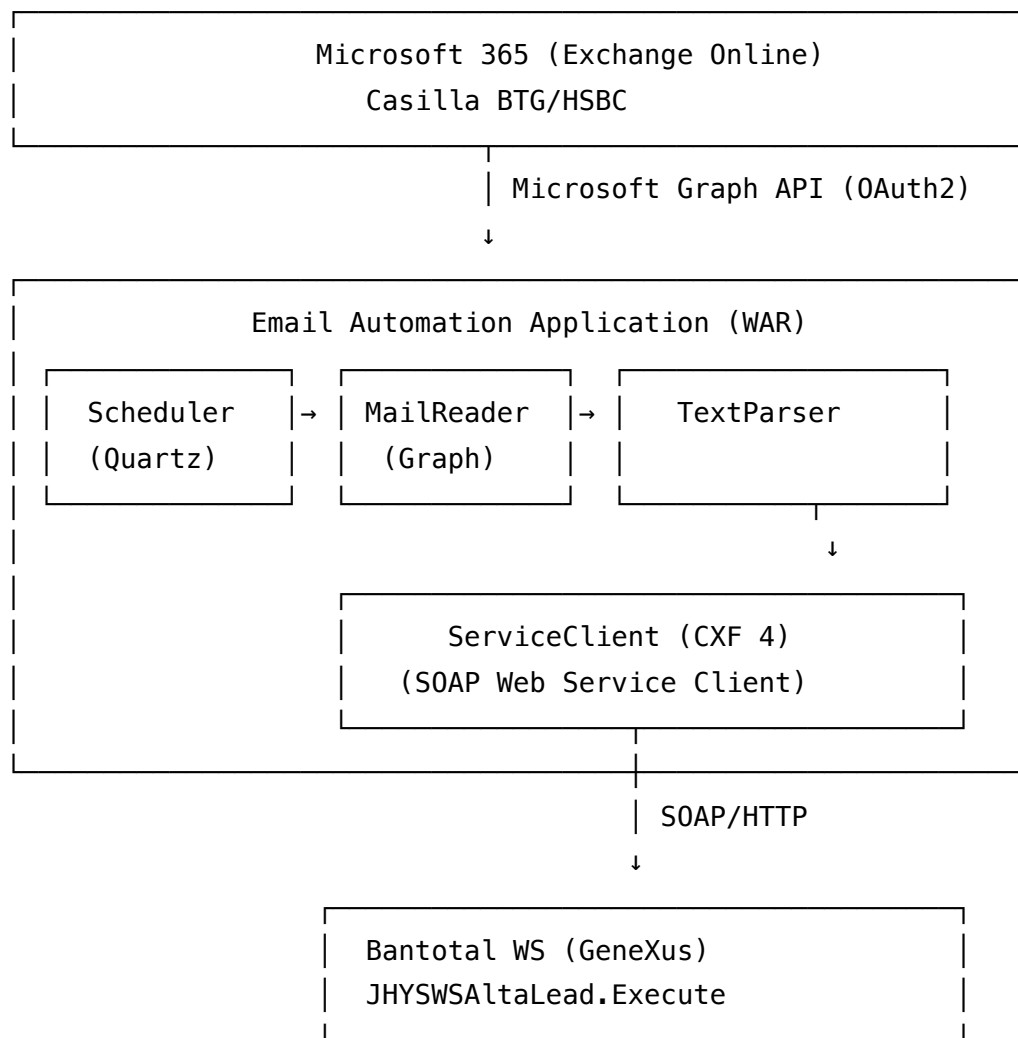
## Arquitectura

### Tecnologías Principales

- **Java 21 / Jakarta EE 11** (WAR deployable en Tomcat 11)

- **CDI (Weld 6)** — Inyección de dependencias
- **JAX-RS (Jersey 3.1)** — Servicios REST
- **Quartz Scheduler** — Ejecución programada
- **Microsoft Graph SDK v6 + azure-identity** — Lectura de correo
- **Apache CXF 4** — Cliente SOAP generado desde WSDL
- **Jsoup** — Parseo de contenido HTML
- **Log4j 2** — Logging

## Arquitectura del Sistema



# Componentes Principales

## 1. Scheduler (Quartz) — `com.globesoftware.email_automation.schedule`

- **Init.java** : `ServletContextListener` que inicializa el scheduler Quartz al arrancar. Lee la expresión `cron` ; si está vacía, no se agenda ejecución.
- **EjecucionJob.java** : job Quartz ( `@DisallowConcurrentExecution` ). Obtiene `MailAutomationBean` del contenedor CDI con `CDI.current()` y ejecuta `process()` .
- **Config.java** : gestión de configuración. Lee el archivo indicado en la system property `email-automation.config.file` . Métodos para `String/Integer/Boolean/Arrays`.

## 2. Procesamiento de Emails

- **MailAutomationBean.java** : bean CDI que orquesta; delega en `MailReader` .
- **MailReader.java** — lee correos no leídos vía **Microsoft Graph**:
  - i. Autentica con `ClientSecretCredential` → `GraphServiceClient` (client credentials).
  - ii. Lista no leídos:

```
GET /users/{email}/mailFolders/{folder}/messages?$filter=isRead eq false
```

(más `from/emailAddress/address` si hay filtro de remitente), con `$select` y paginación por `@odata.nextLink` .
  - iii. Toma el body directo ( `message.getBody()` ); si es HTML lo limpia con Jsoup.
  - iv. Parsea con `TextParser` , envía con `ServiceClient` , y marca leído/no leído con `PATCH /users/{email}/messages/{id}` ( `isRead` ).
    - **Modo mock**: si `graph_base_url` está seteado, usa un token estático (sin Azure AD) y apunta el SDK a ese endpoint — usado por los tests con WireMock.
- **TextParser.java** — extrae pares clave-valor del texto. Dos modos:
  - **Multiline**: Token: Valor por línea (autodetecta si en realidad es una sola línea).
  - **Singleline**: para HTML convertido a texto; `remove_trailer_from` permite cortar el final.

## 3. Integración SOAP

- **ServiceClient.java** : cliente del WS SOAP generado desde WSDL. Lee `endpoint` de config, registra `SoapLogHandler` , arma `SdtCampos` , llama `Execute` con `tipo` , retorna `errorId` (0 = éxito).
- **Clases generadas** ( `gen-src/` ): generadas por CXF 4 desde `wsdl/WS_nuevo.xml` (namespaces `jakarta` ). `JHYSWSAltaLead` , `JHYSWSAltaLeadSoapPort` , `JHYSWSAltaLeadExecute(Response)` , `SdtCampos` , `WsscoringSdtCamposSdtCamposItem` .

## 4. Servicios REST (manuales)

- `/rest/ejecutar` ( `Ejecutar.java` ): dispara el proceso de lectura en un thread separado.
- `/rest/testws` ( `Testws.java` ): prueba el WS SOAP con datos hardcodedos.

## 5. Logging

- `Logger.java` : wrapper sobre Log4j 2 (config en `email-automation-log4j.xml` ).
- `SoapLogHandler.java` : handler JAX-WS que loguea request/response SOAP.

## Configuración

System property obligatoria: `email-automation.config.file` (path al `.properties` ).

Plantilla completa en `config/email-automation.example.properties` .

```

# Scheduler (vacío = solo manual por REST)
cron=0 0/5 * * * ?

# Microsoft Graph (App Registration con Mail.ReadWrite)
graph_tenant_id=xxxxxxxx-xxxx-xxxx-xxxx-xxxxxxxxxxxxxx
graph_client_id=xxxxxxxx-xxxx-xxxx-xxxx-xxxxxxxxxxxxxx
graph_client_secret=xxxxxxxxxxxxxxxxxxxxxxxxxxxxxx
graph_user_email=casilla@btg.com
# Override del endpoint Graph – SÓLO testing (WireMock). Vacío en producción.
graph_base_url=

# Carpeta(s) y filtro de remitente (opcional)
mail_folders=inbox
sender=

# Parseo
tokens=Nombre del cliente,Celular del cliente,Mail,Cedula,Telefono del cual se contacto
remove_tokens=<mailto:,mailto:
separator=:
trim_values=true
remove_trailer_from=Aviso de confidencialidad

# Comportamiento y SOAP
on_error_mail_read=false
endpoint=http://server:port/ti-ws-scoring/servlet/com.dlya.bantotal.ajhyswsaltalead
tipo=LEAD

```

## Flujo de Procesamiento

1. **Scheduler o REST** dispara la ejecución.
2. **MailReader** lista los no leídos de cada folder vía Graph (con filtro opcional por remitente).
3. Por cada mensaje: extrae el body (text/html), **TextParser** saca los campos, **ServiceClient** los envía al WS SOAP.
4. Si `errorId=0` → marca leído ( `isRead=true` ); si no → según `on_error_mail_read` .

## Build y ejecución

Política **cero-runtimes**: el developer solo necesita **Docker**. El build/run se orquesta desde el repo de infra hermano `docker-hsbc-email-autom`

(build-container Maven con Zulu 21 + runtime Tomcat 11).

```
# clonar ambos repos como hermanos:
#   .../email-automation/hsbc-email-autom          (este repo)
#   .../email-automation/docker-hsbc-email-autom (infra)

cd ../docker-hsbc-email-autom
make up          # build-container + mock SOAP/Graph (WireMock)
make build       # compila el WAR dentro del container → target/email-automation.war
make run         # levanta Tomcat 11 / Zulu 21 con el WAR
make logs-app    # logs en vivo

# app: http://localhost:8080/email-automation
```

make help lista todos los comandos. Detalle en el README del repo de infra.

## Deployment en producción

El WAR ( target/email-automation.war ) se despliega en Tomcat 11 con:

```
CATALINA_OPTS="$CATALINA_OPTS -Demail-automation.config.file=/path/to/email-automation.
```

## Testing

Suite de **17 tests automáticos** (unitarios + integración con WireMock embebido, sin servicios externos):

```
cd ../docker-hsbc-email-autom
make build-tests    # Tests run: 17, Failures: 0, Errors: 0
```

Cubre el flujo Graph → parser → SOAP → marcado, con escenarios de error (401, 429, error SOAP), paginación, HTML, filtro por remitente. Detalle y cómo correr tests individuales en [docs/VALIDACION.md](#) ("Cómo correr los tests" y "Fase 6b").

Endpoints REST para disparo/prueba manual:

```
curl http://localhost:8080/email-automation/rest/ejecutar # dispara el proceso
curl http://localhost:8080/email-automation/rest/testws   # prueba el WS SOAP
```

# Dependencias Clave

- Weld 6.0.0 (CDI / Jakarta EE 11)
- Jersey 3.1.9 (JAX-RS) + jersey-cdi1x
- Apache CXF 4.2.2 (SOAP) + saaj-impl 3.0.4
- Microsoft Graph SDK 6.65.0 + azure-identity 1.14.2
- jakarta.servlet-api 6.1.0 (provista por Tomcat)
- Quartz 2.3.2 (Scheduler)
- Jsoup 1.18.1 (HTML)
- Log4j 2.24.3

# Documentación

- [docs/QUICK\\_START.md](#) — arranque rápido en local (solo Docker) y prueba con mocks.
- [docs/DEMO.md](#) — guion de demo para el cliente (flujo end-to-end sobre mocks).
- [docs/PUESTA\\_EN\\_PRODUCCION.md](#) — qué pedir a BTG y cómo configurar Graph real.
- [docs/DOCKER.md](#) — entorno Docker (repo de infra), runtime Tomcat 11 / Zulu 21.
- [docs/migracion\\_graph\\_api.md](#) — plan de migración (alcance, fases).
- [docs/VALIDACION.md](#) — cómo validar cada fase y correr los tests.
- [docs/Avance.md](#) — bitácora del trabajo y decisiones.
- [docs/DOCUMENTACION\\_INDEX.md](#) — índice de docs.